

GSoC21 W7: LFortran Workflow Basics

Rohit Goswami · 2021-07-23 · gsoc21 · fortran · ramblings

Revisiting `minidftatom` and a birds eye view of `lfortran`

Background

Serialized update for the [2021 Google Summer of Code](#) under the [fortran-lang organization](#), mentored by [Ondrej Certik](#).

Logistics

- Met with Ondrej on Tuesday
 - Went over the debugging workflow
 - Discussed the design of the `SymbolTableVisitor` and `BodyVisitor`
 - Fixed a symbol scoping bug and discussed implementing `value` for `kind`
- Kept in touch asynchronously over Zulip and MRs

Overview

This week focused on implementing features and cleaning up parts of the project.

New Merge Requests

Cleanup Integer Binary Operations (1045)

There was an extraneous check for something the `LFORTRAN_ASSERT` would catch

Values for Real Binary Operations (1080)

An implementation, which is similar conceptually and in implementation to the Integer Value MR

Symbol Table Scope Fix for Function Calls (1087)

A bug which haunted me for a few weeks turned out to have a similar fix in the `BodyVisitor`

Intrinsic Issue Template (1088)

I am rather fond of a particular style of opening missing intrinsic issues, and this codifies it for future use

Draft: ci: Use Ninja everywhere (1085)

A draft MR to return to after updating the CI sometime

Freshly Assigned Issues

Suggestion to refactor `ast_to_asr` (467)

In particular this has to do with redundancies (of sort) in the `SymbolTable` and `Body` visitors

Implement the `sqrt` intrinsic (468)

Or at-least, for this issue, recognize it as a valid intrinsic

Implement the `where` conditional (475)

Should be implemented along the same lines as the other conditionals

Semantic error: Variable not declared (476)

Another fascinating episode of “bug detective”, this one might be more of a red herring

Semantics: Minidftatom Math (477)

Collection of intrinsic math functions to be implemented at the ASR level

Additional Tasks

Perhaps adding some more templates would make things easier (for me and other contributors). I intend to update the [CI images](#) to leverage `ninja`.

Misc Logs

Build Times

`ninja` is fantastic. The `make` and `cmake` based workflow I was using often had me stalling for upto half a minute.

Table 1: Not a rigorous benchmark at all, estimates in seconds

Generator	Cold Build	Rebuild
“Unix Makefiles”	17	16
“Ninja”	11	10

Stacktrace Debugging

Ondrej mentioned in the past that he does not use a debugger, and in our meeting we went over extracting information from the `stacktrace` produced by `lfortran`. I have personally found it to be rather verbose, but then he pointed out that I could ignore the generated lines from `ast.h`. That made things a lot easier.

Scoping Rules

- Intrinsics are exposed to the global scope, which has a `nullptr` parent
- Inside a sub-module, we need to shift the scope, which was done for the `BodyVisitor` but not for the `SymbolTableVisitor`

Conclusions

This week was possibly more productive in terms of code than any before. For that, I mostly have to thank that sudden flash of **understanding** which comes from working on a code-base for weeks (with a fantastic guide). Next week should see the rest of `expr_value` be completed, and I intend to try my hand at the LLVM backend to make it use `expr_value` when possible. Naturally, this will be done in tandem with the `minidftatom` issues, after-all it is back to basics in the sense that progress resumed on working through the main `minidftatom` roadmap as well.